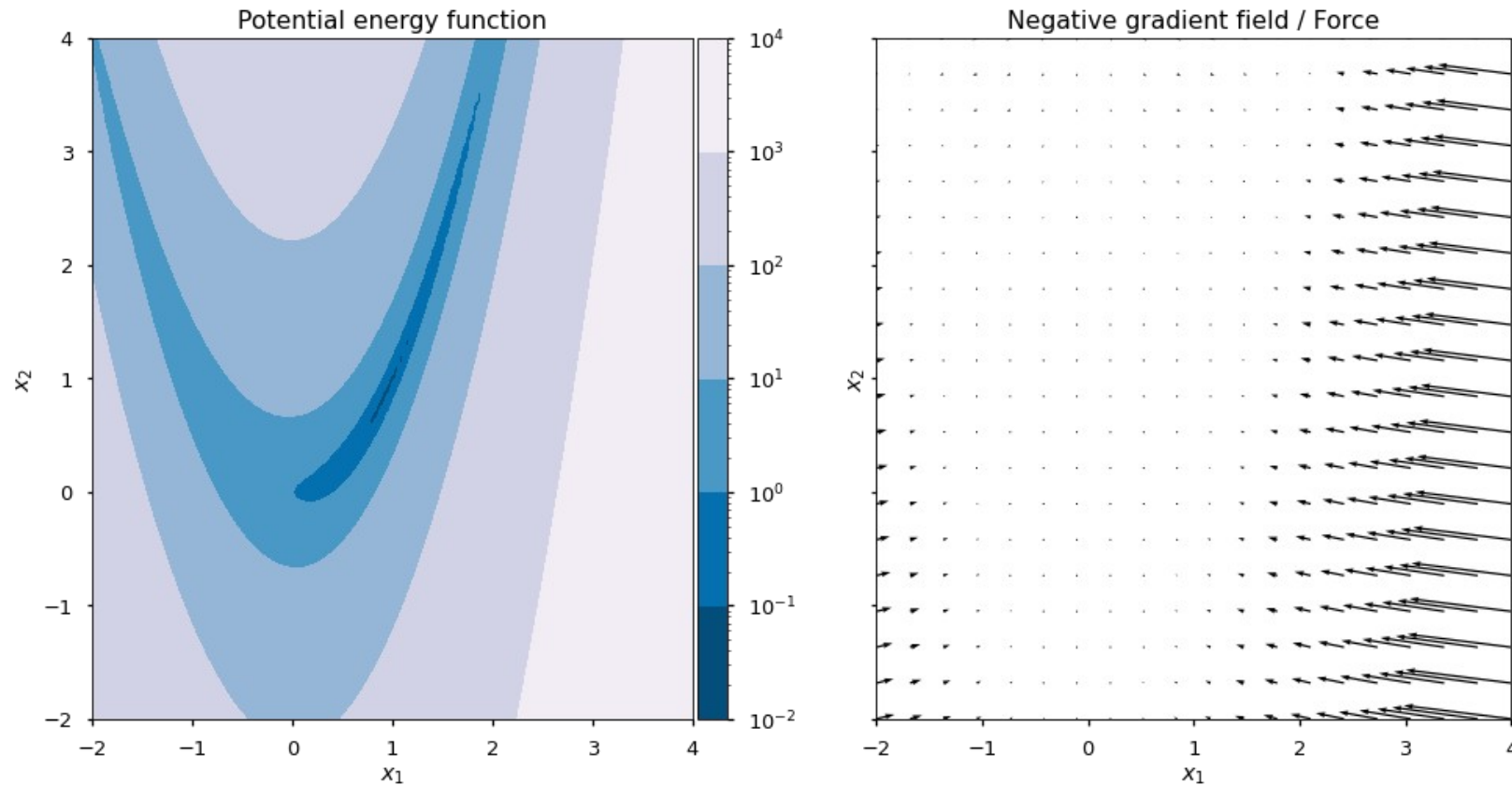
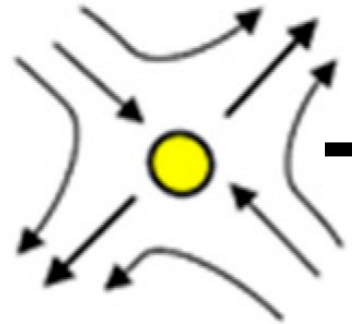


Example loss: Rosenbrock function

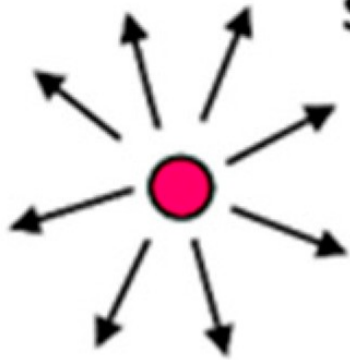


The local minimum is an “*attracting fixed point*” of the dynamics

Repeller / unstable fixed point

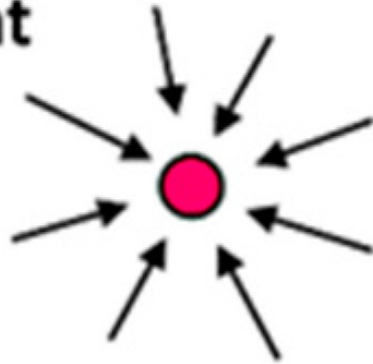


saddle point



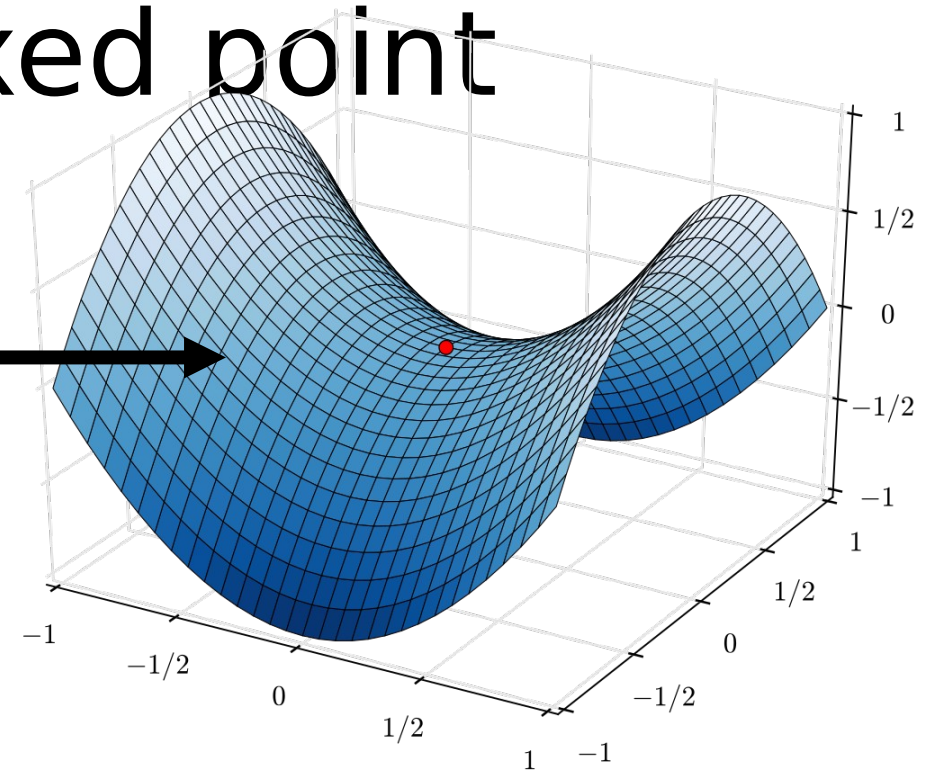
unstable node

repelling

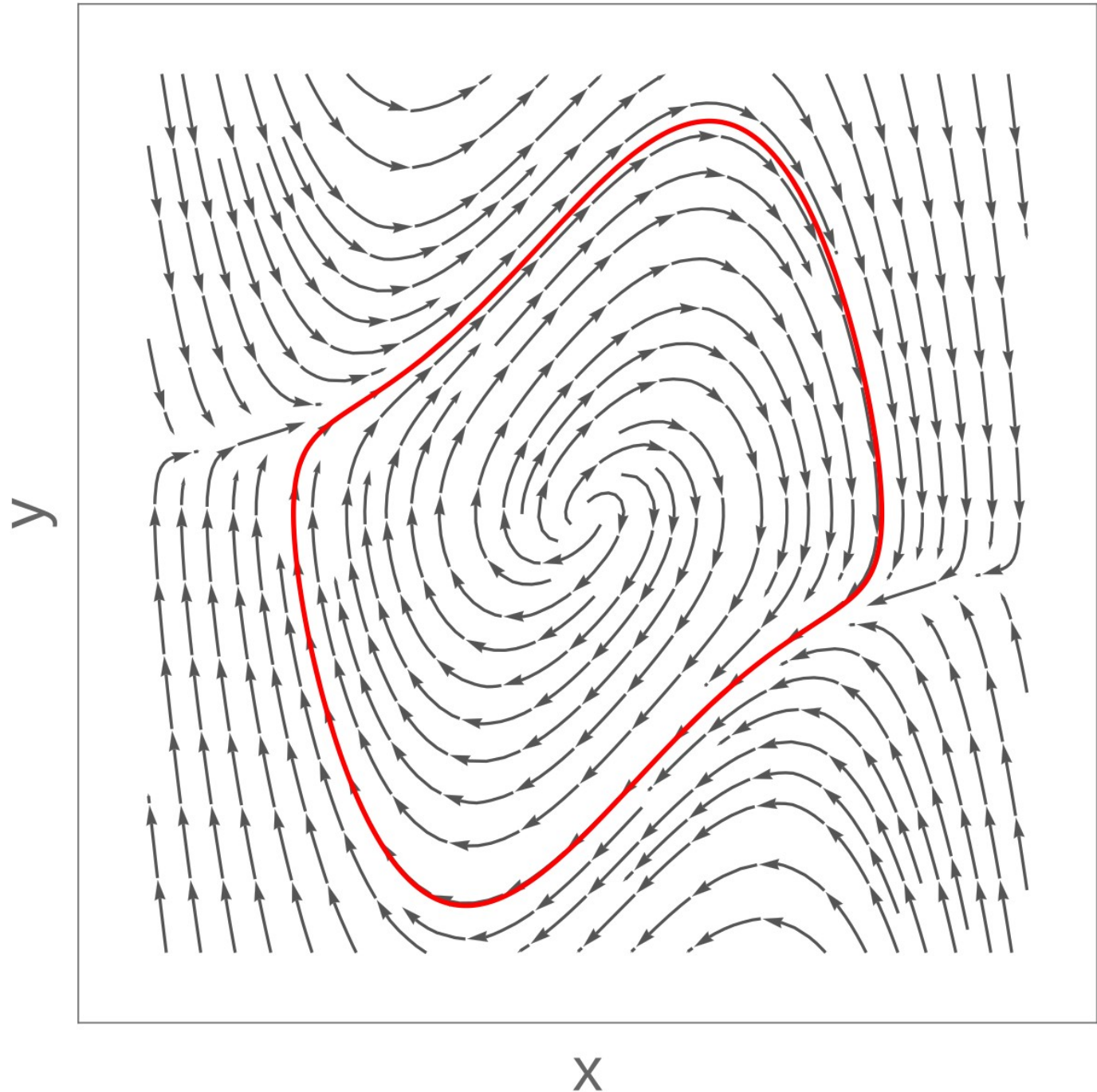


stable node

attracting



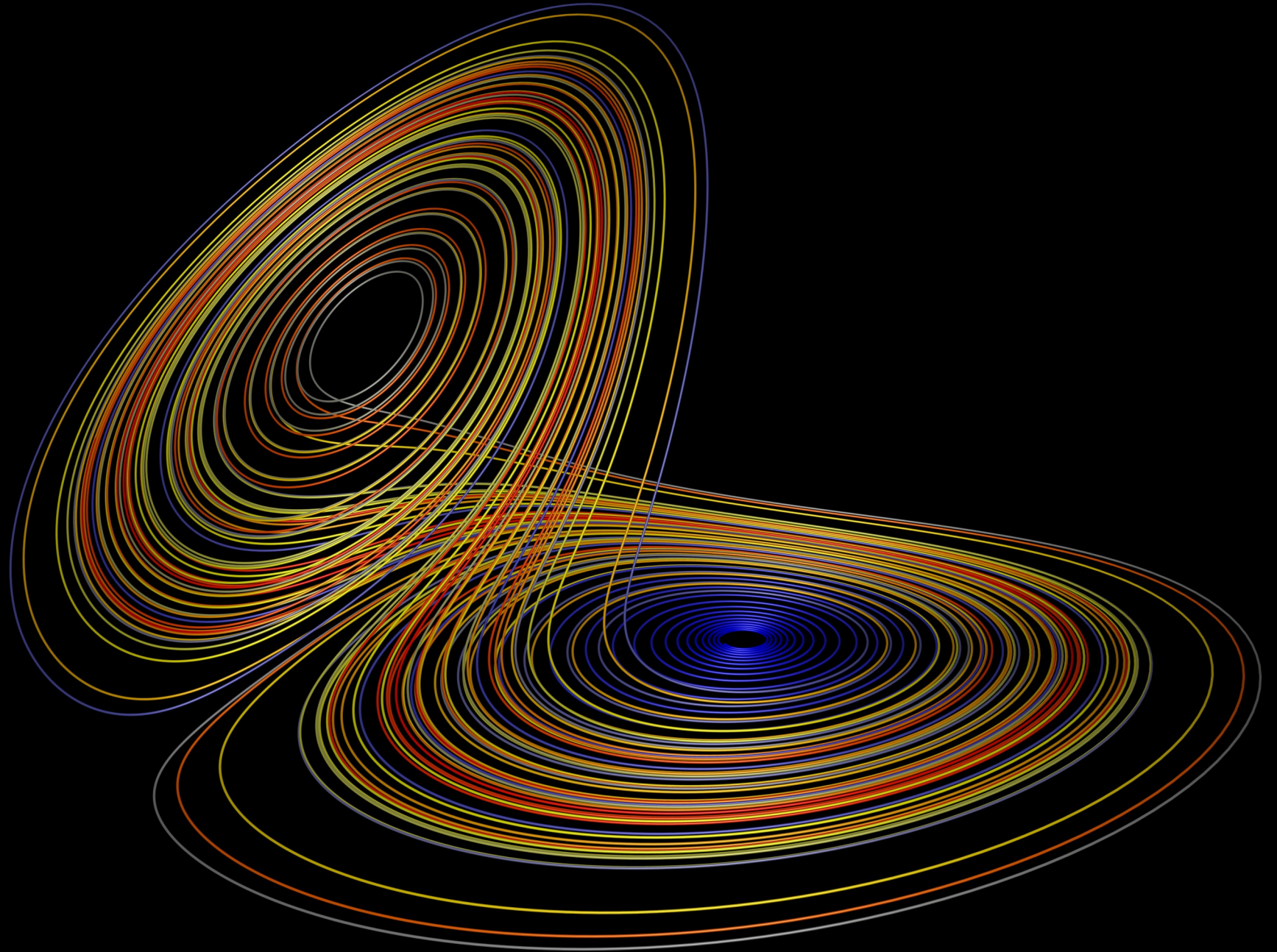
Limit cycle



<https://arxiv.org/abs/1710.11029>

SGD limit cycles for deep nets?
(I guess follow-up work said no)

Chaos – strange attracto r



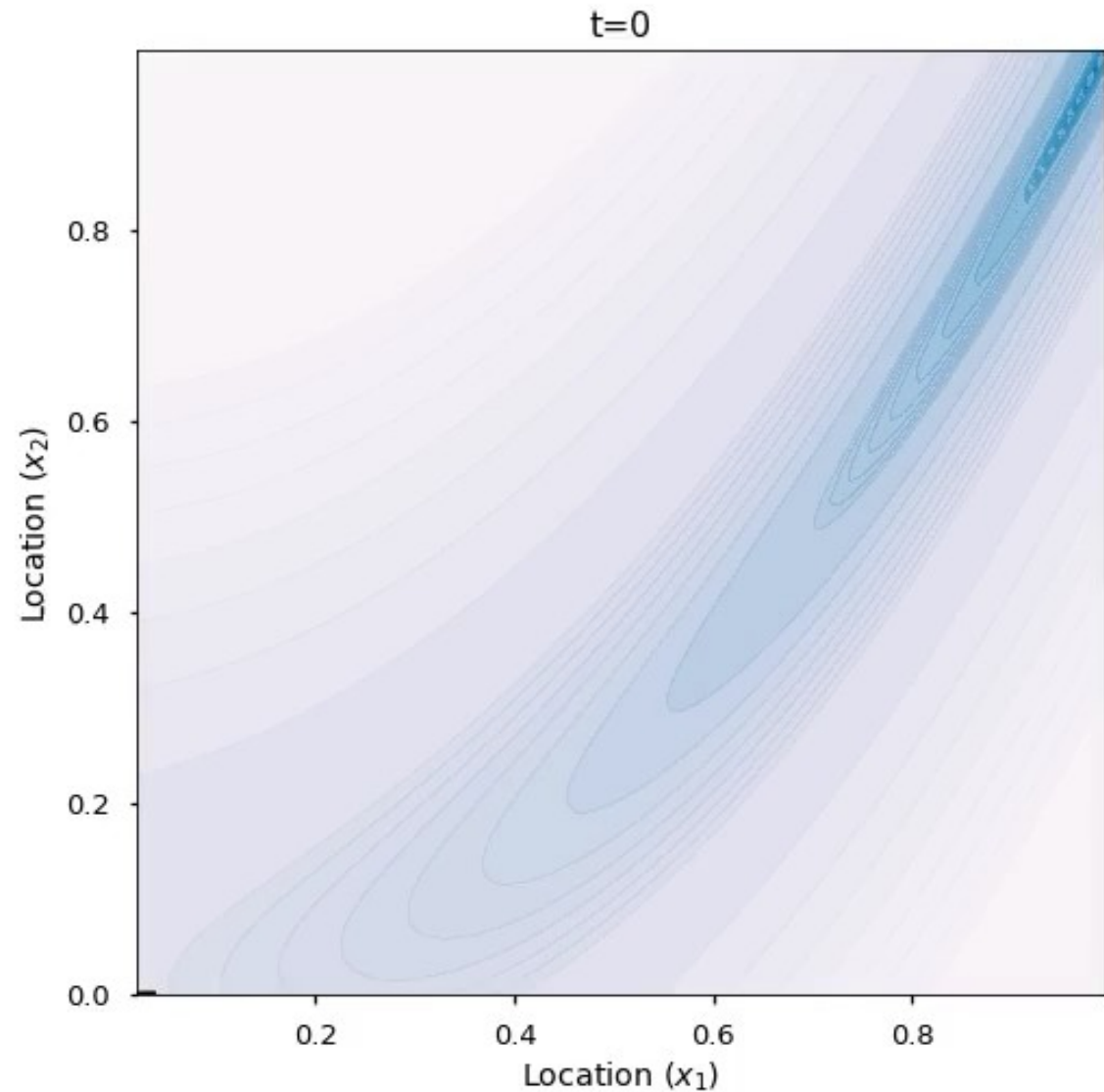
Back to physics

Newton:

Two dots!

One dot!

Classic “one
dot” slow
down
(gradient
descent)



The difference has to do with *momentum*

“Momentum” in ML is really useful for good results with SGD

CIFAR-100 hyper-parameter tuning experiment: <https://github.com/meliketoy/wide-resnet.pytorch>

epoch		learning rate	weight decay	Optimizer	Momentum	Nesterov	
0 ~ 60		0.1	0.0005	Momentum	0.9	true	
61 ~ 120		0.02	0.0005	Momentum	0.9	true	
121 ~ 160		0.004	0.0005	Momentum	0.9	true	
161 ~ 200		0.0008	0.0005	Momentum	0.9	true	
network	dropout	preprocess	GPU:0	GPU:1	per epoch	Top1 acc(%)	Top5 acc(%)
wide-resnet 28x20	0.3	meanstd	8.13G	6.93G	4 min 05 sec	82.45	96.11

Momentum - Newton to Hamilton

-

Beauty of Hamilton's formulation

•

$$\underset{\text{Hamiltonian}}{H(\underset{\text{“position”},}{\underset{\text{Potential}}{\text{energy}}}{x}, \underset{\text{Kinetic}}{\underset{\text{energy}}{\text{“momentum” or “velocity”}}}{v})} = \underset{\text{Potential}}{\underset{\text{energy}}{E(\underset{\text{“position”},}{\underset{\text{Potential}}{\text{energy}}}{x})}} + \underset{\text{Kinetic}}{\underset{\text{energy}}{K(\underset{\text{“momentum” or “velocity”}}{\underset{\text{Kinetic}}{\text{energy}}}{v})}}$$

Hamiltonian explicitly connects conservation laws and dynamics

Hamiltonian dynamics

Hamiltonian

Potential
energy

Kinetic
energy

How do we know
the energy is
conserved for
these dynamics?

$$\dot{H} = dH(\mathbf{x}, \mathbf{v})/dt = \frac{\partial H}{\partial \mathbf{x}} \cdot \frac{d\mathbf{x}}{dt} + \frac{\partial H}{\partial \mathbf{v}} \cdot \frac{d\mathbf{v}}{dt}$$

$$\dot{H} = \frac{\partial H}{\partial \mathbf{x}} \cdot \dot{\mathbf{x}} + \frac{\partial H}{\partial \mathbf{v}} \cdot \dot{\mathbf{v}}$$

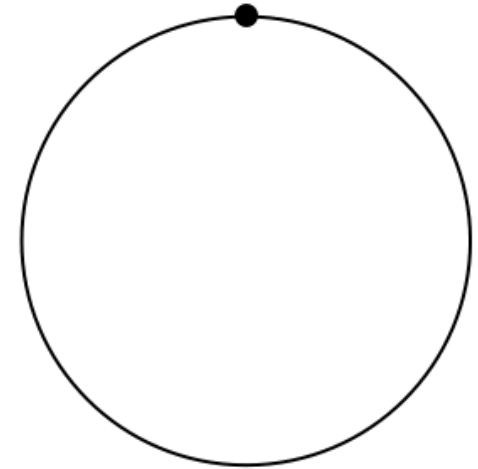
$$= \frac{\partial H}{\partial \mathbf{x}} \cdot \frac{\partial H}{\partial \mathbf{v}} + \frac{\partial H}{\partial \mathbf{v}} \cdot \left(-\frac{\partial H}{\partial \mathbf{x}}\right)$$

$$\dot{H} = 0$$

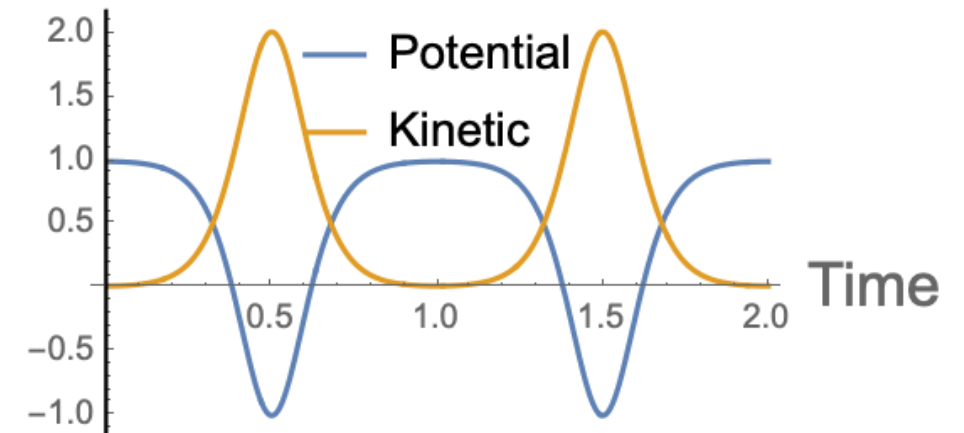
Roller coaster momentum and conservation

This momentum isn't good for optimization at all.

Newtonian Momentum

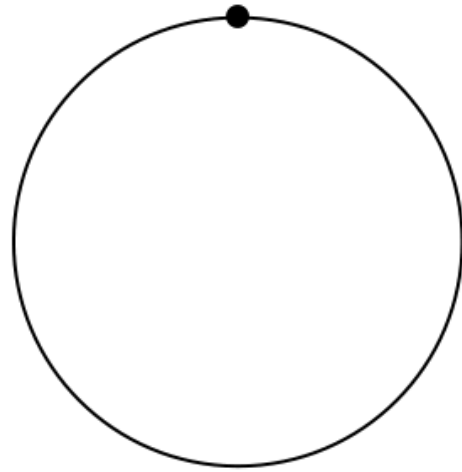


Energy

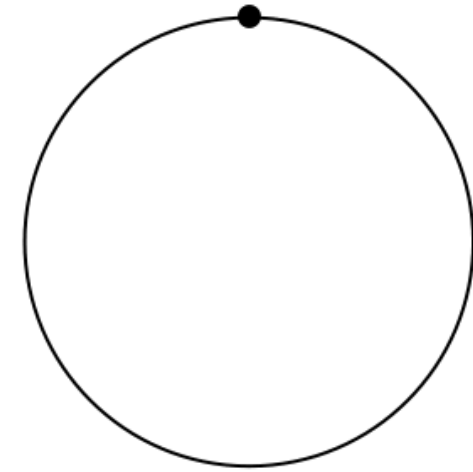


(BTW: the sampling paper I mentioned uses a non-Newtonian momentum
<https://arxiv.org/abs/2111.02434>
I originally made this figure for that.)

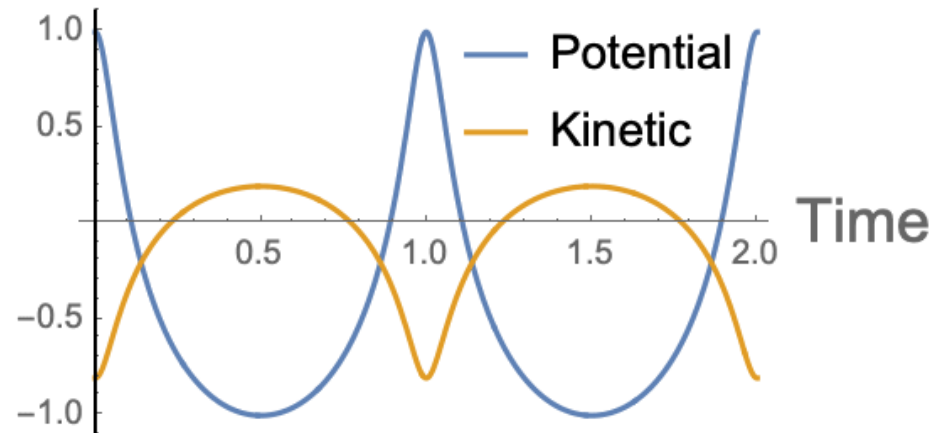
ESH momentum



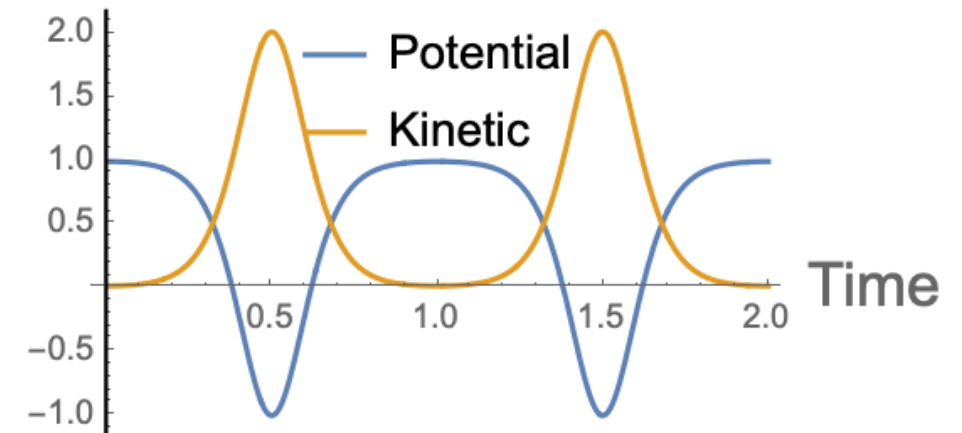
Newtonian Momentum



Energy



Energy



“Momentum” in ML is momentum + friction

Classical dynamics
with momentum

Classical dynamics
with momentum and friction

-

“Momentum” in ML is momentum + friction

Classical dynamics
with momentum

Classical dynamics
with momentum and friction

-

On the importance of initialization and momentum in deep learning

Ilya Sutskever¹

ILYASU@GOOGLE.COM

2. Momentum and Nesterov's Accelerated Gradient

The momentum method (Polyak, 1964), which we refer to as classical momentum (CM), is a technique for accelerating gradient descent that accumulates a velocity vector in directions of persistent reduction in the objective across iterations. Given an objective function $f(\theta)$ to be minimized, classical momentum is given by:

$$v_{t+1} = \mu v_t - \varepsilon \nabla f(\theta_t) \quad (1)$$

$$\theta_{t+1} = \theta_t + v_{t+1} \quad (2)$$

On the importance of initialization and momentum in deep learning

Ilya Sutskever¹

ILYASU@GOOGLE.COM

$$v_{t+1} = \mu v_t - \varepsilon \nabla f(\theta_t) \quad (1)$$

$$\theta_{t+1} = \theta_t + v_{t+1} \quad (2)$$

- Classical momentum

cally, as shown in the appendix, the NAG update may be rewritten as:

$$v_{t+1} = \mu v_t - \varepsilon \nabla f(\theta_t + \boxed{\mu v_t}) \quad (3)$$

$$\theta_{t+1} = \theta_t + v_{t+1} \quad (4)$$

- Nesterov accelerated gradient (NAG) has optimal convergence for convex optimization

Helpful (?) illustration from Sutskever et al.

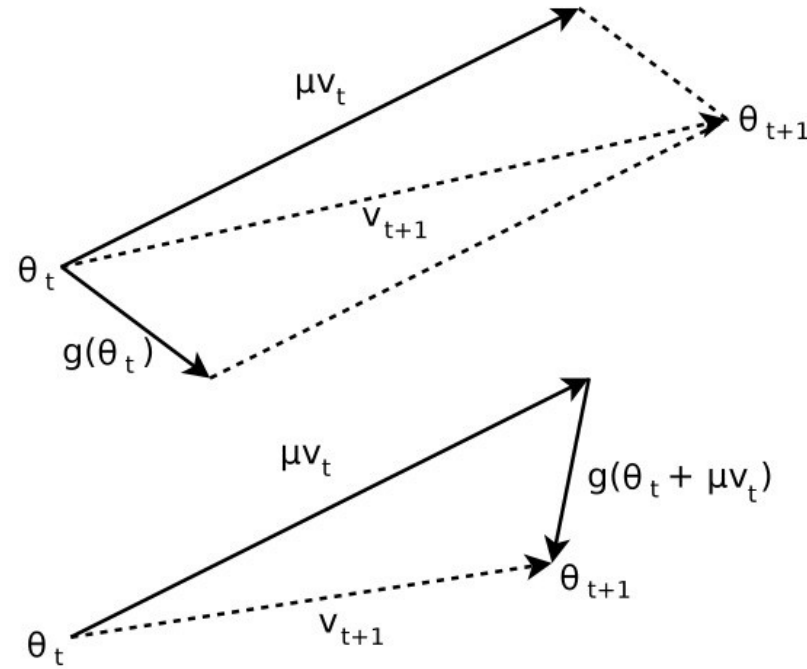


Figure 1. **(Top)** Classical Momentum **(Bottom)** Nesterov Accelerated Gradient

More helpful (to me)

$$\ddot{X} + \frac{3}{t}\dot{X} + \nabla f(X) = 0$$

A Differential Equation for Modeling Nesterov's Accelerated Gradient Method: Theory and Insights

Weijie Su

*Department of Statistics
University of Pennsylvania
Philadelphia, PA 19104, USA*

SUW@WHARTON.UPENN.EDU

Stephen Boyd

*Department of Electrical Engineering
Stanford University
Stanford, CA 94305, USA*

BOYD@STANFORD.EDU

Emmanuel J. Candès

*Departments of Statistics and Mathematics
Stanford University
Stanford, CA 94305, USA*

CANDES@STANFORD.EDU

Editor: Yoram Singer

Abstract

We derive a second-order ordinary differential equation (ODE) which is the limit of Nesterov's accelerated gradient method. This ODE exhibits approximate equivalence to Nesterov's scheme and thus can serve as a tool for analysis. We show that the continuous time ODE allows for a better understanding of Nesterov's scheme. As a byproduct, we obtain a family of schemes with similar convergence rates. The ODE interpretation also suggests restarting Nesterov's scheme leading to an algorithm, which can be rigorously proven to converge at a linear rate whenever the objective is strongly convex.

Keywords: Nesterov's accelerated scheme, convex optimization, first-order methods, differential equation, restarting

Warning – parameters of momentum implementation don't match across frameworks

- <https://pytorch.org/docs/stable/generated/torch.optim.SGD.html>

• NOTE

The implementation of SGD with Momentum/Nesterov subtly differs from Sutskever et. al. and implementations in some other frameworks.

Considering the specific case of Momentum, the update can be written as

$$v_{t+1} = \mu * v_t + g_{t+1},$$

$$p_{t+1} = p_t - \text{lr} * v_{t+1},$$

where p , g , v and μ denote the parameters, gradient, velocity, and momentum respectively.

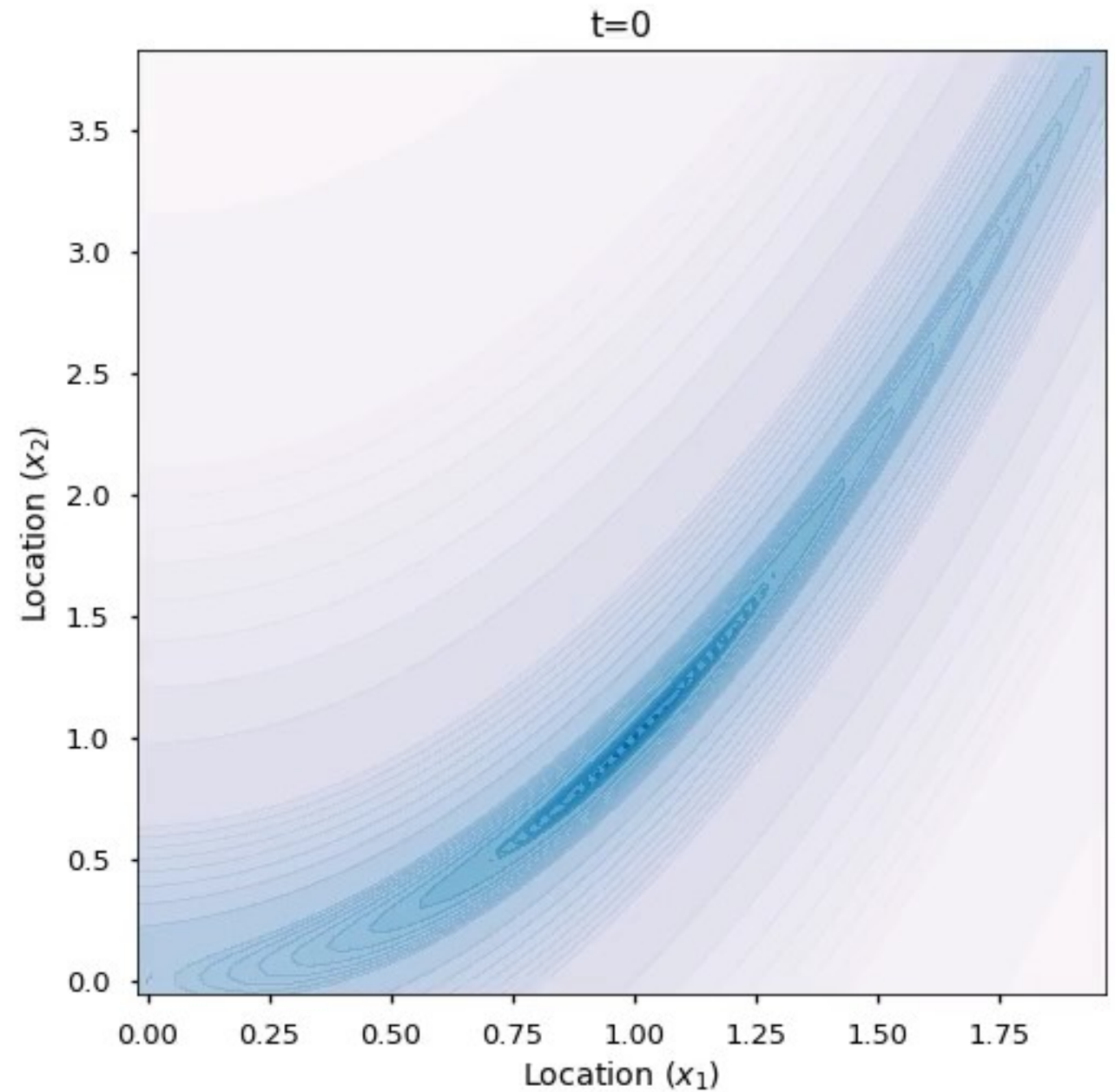
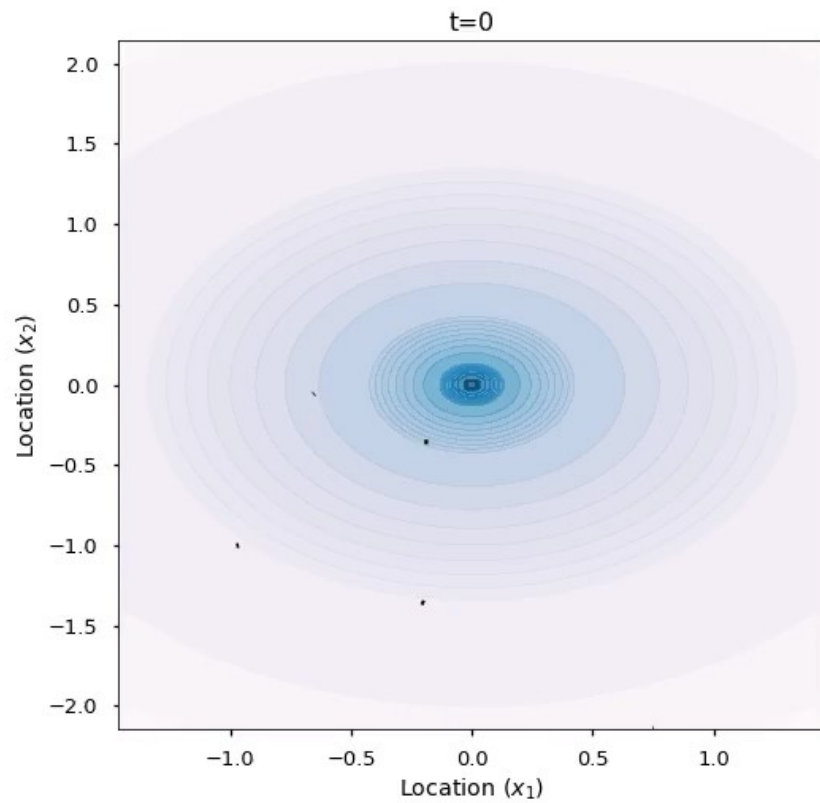
This is in contrast to Sutskever et. al. and other frameworks which employ an update of the form

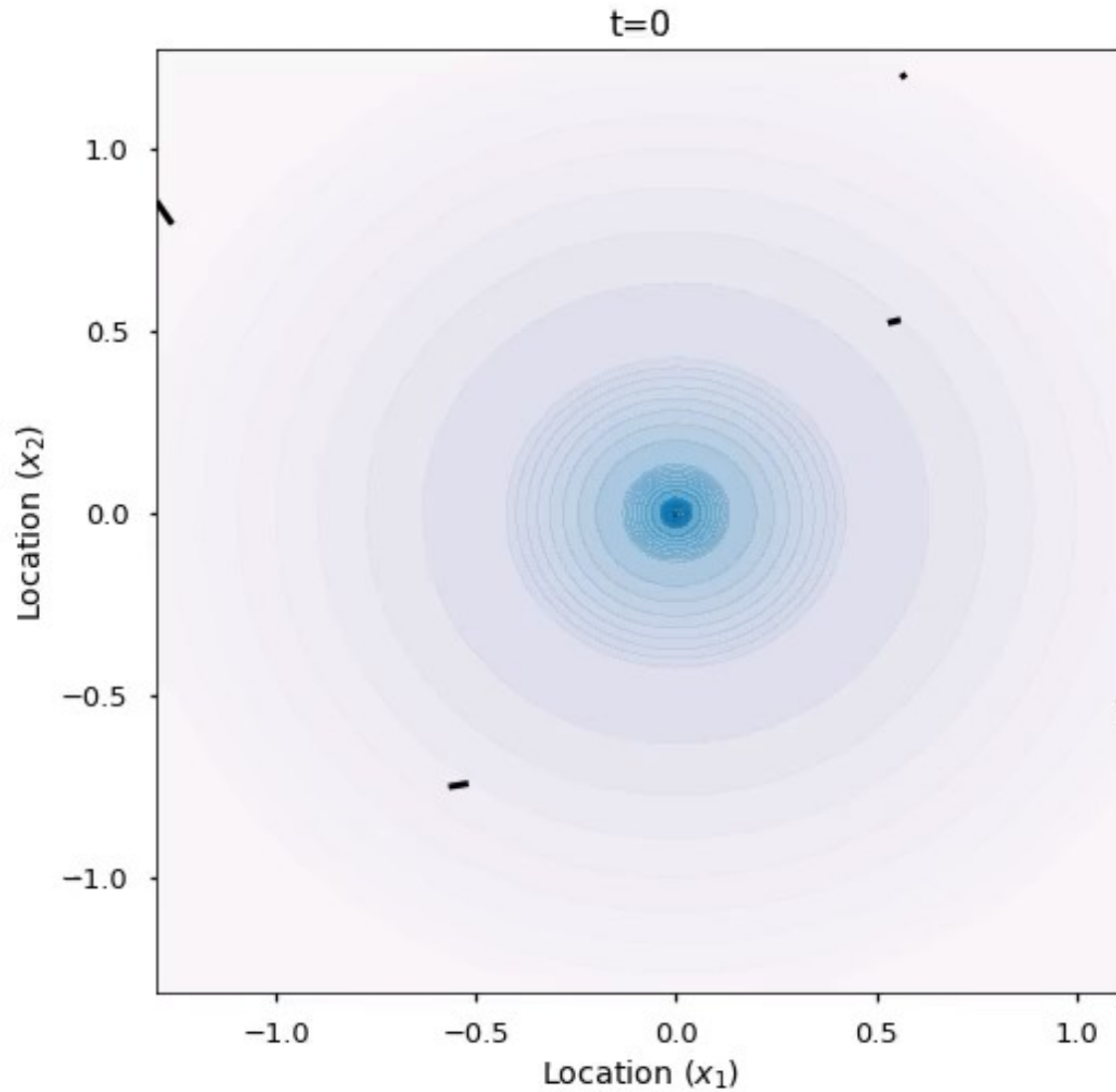
$$v_{t+1} = \mu * v_t + \text{lr} * g_{t+1},$$

$$p_{t+1} = p_t - v_{t+1}.$$

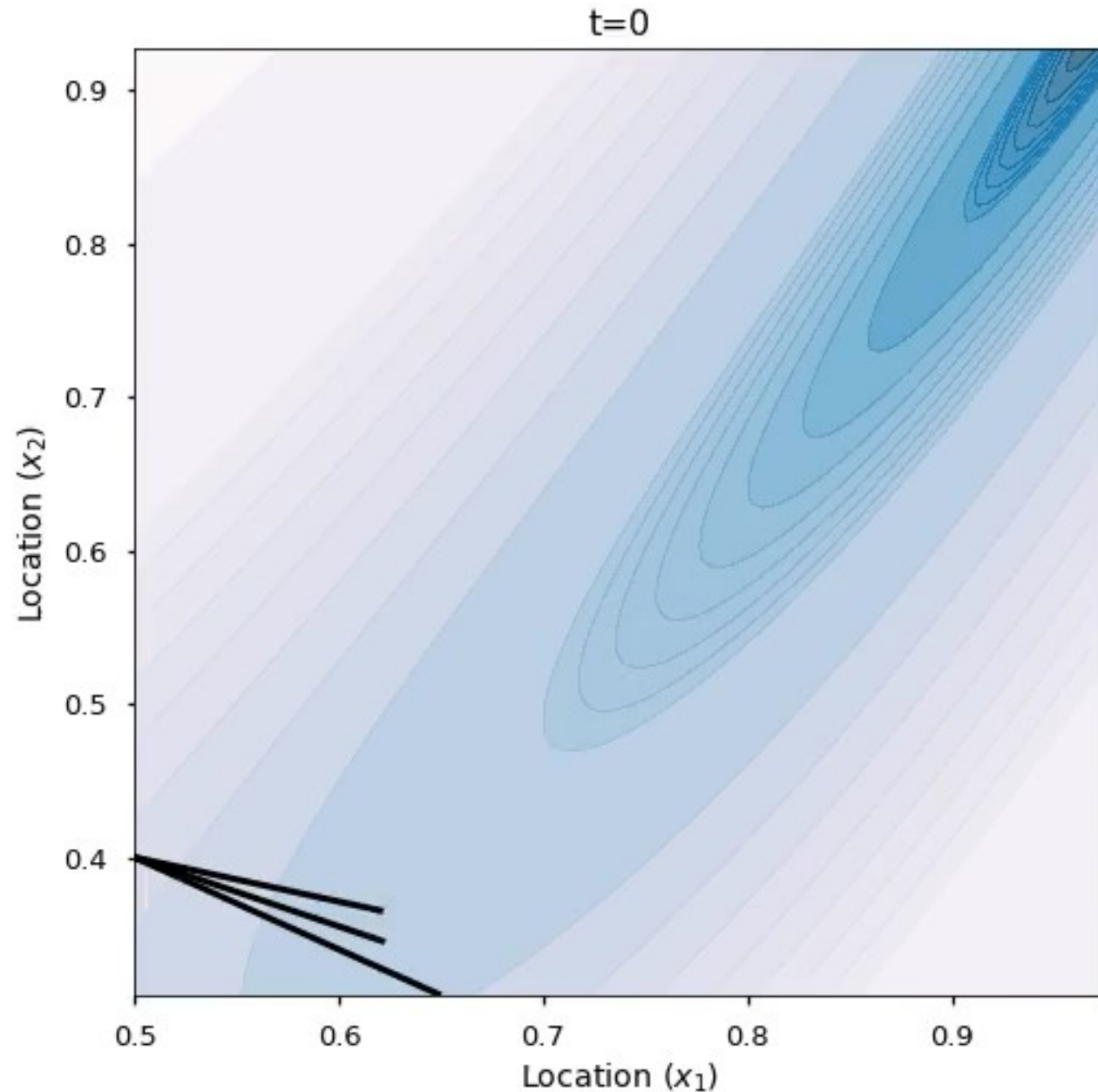
The Nesterov version is analogously modified.

Hamiltonian dynamics (momentum but no friction)





- Hamiltonian dynamics with friction
- Classic (static) friction slows things down too much!
- (Hence default is “nesterov” momentum)



- Using large step size is implicitly like adding acceleration, but can lead to numerical errors
- Noether dynamics paper, NeurIPS 2021

Wednesday: SGD and why it really works

-

Sum of iid random variables

For large enough batch size,
central limit theorem describes
distribution

Wednesday: SGD and why it really works

-